Mid-Term Examination

Building AI Agents

Set-B

Maximum marks: 100 **Duration: 120 Minutes**

Instructions

1. *This is a **single problem statement** exam. You must complete the full case study described below.*
2. *You are allowed to use **AI coding assistants** — Cursor, VS Code with Copilot, or Google Colab AI. Usage of AI tools is encouraged and expected.*
3. ***You may use any Python library: LangChain, CrewAI, a2a, OpenAI SDK, or any other agentic framework.***
4. ***Your solution must run 100% locally. No cloud deployment, no external hosting. Use Streamlit for UI.***
5. *Push all code to your GitHub repository. The evaluator will run: python run.py OR python start.py to launch your solution.*
6. *Submission window: Your last commit must be within the 2-hour exam window (±10 minutes). Commits outside this window will NOT be evaluated.*
7. *Include a README.md in your repo with: setup steps, pip install requirements, and your .env variables format (do not commit actual API keys).*
8. *The dataset file (creditsage_loan_applications.csv) will be provided to you at the start of the exam. Place it in your project root.*

Rubrics

Criteria	Marks	What Will Be Evaluated
Functional Correctness	**40**	Does the agent work end-to-end? Eligibility logic correct? EMI formula accurate? Memory functional? No crashes during evaluator's test session? All 5 tools demonstrably callable?
Agentic Design & Code Architecture	**25**	Is the Router Pattern correctly implemented using LLM-based intent classification (not keyword matching)? Are tools properly defined with schemas? Is code modular and readable?
LLM & Prompt Engineering	**20**	Are system prompts well-crafted? Are LLM parameters (temperature, top_p, max_tokens) set thoughtfully and commented? Are tool schemas clearly described? Do prompts guide the LLM to use tools vs. answering from

		knowledge?
Business Understanding & Demo Quality	**15**	Can the student explain the business value of this agent vs. a human advisor? Is the Streamlit UI intuitive? Does the README clearly explain setup? Are code comments meaningful?
TOTAL	**100**	

1. Company Background & Business Problem

RetailMind Analytics is an early-stage B2B SaaS startup based in Mumbai. They build AI-powered product intelligence tools for D2C (Direct-to-Consumer) brands selling on platforms like Shopify, Myntra, and their own websites. RetailMind's platform helps product managers answer one question instantly: "What is happening in my catalog right now, and what should I do about it?"

RetailMind's flagship client is StyleCraft, a D2C fashion brand with over 80 active SKUs across 5 categories: Tops, Dresses, Bottoms, Outerwear, and Accessories. StyleCraft's product manager — Priya Mehta — currently spends every Monday morning manually pulling sales data, checking inventory levels, reading customer reviews, and identifying pricing issues. This weekly analysis takes 4–5 hours and is mostly reactive — by the time Priya spots a stockout risk or a pricing gap, it's often too late.

RetailMind's CEO wants to showcase an AI agent at their upcoming investor demo that can do everything Priya does in 4–5 hours — but in real time, via a conversational interface. The agent should answer natural language questions about the catalog, proactively surface critical alerts (like imminent stockouts), and generate data-backed merchandising recommendations.

The Business Problem

The Head of Product at RetailMind has defined the following requirements for the AI Agent:

- The agent must answer any natural language question about StyleCraft's product catalog — from 'Which dresses are low on stock?' to 'What's the gross margin on our winter jackets?'
- It must proactively alert the user to critical inventory situations without waiting to be asked.
- It must support multi-turn conversations — Priya should be able to drill down into a category, then ask a follow-up about a specific product, without the agent losing context.
- The agent must intelligently route each query to the right specialised analysis function — inventory queries go to one handler, pricing/margin queries to another, and review/sentiment queries to a third.
- On startup, the agent should proactively scan the catalog and display a daily briefing with the top 3 most urgent issues.

2. Problem Statement

You are a developer at RetailMind tasked with building the Product Intelligence Agent. Build a fully functional, locally-running AI agent system covering all four components below. The agent will be used by StyleCraft's product manager via a Streamlit interface.

Component 1: Tool-Calling Layer (Mandatory)

Your agent must implement the following tools as Python functions callable by the LLM. Define proper tool schemas (function names, descriptions, parameter types) so the LLM can decide when and how to call each tool.

Tool / Function Name	What It Does	Implementation Hint
search_products (query, category=None)	Searches and returns matching products from the CSV based on a text query and optional category filter. Returns product ID, name, category, price, stock, rating.	Use simple string matching or embedding-based similarity. Return top 5 matches as a list of dicts.
get_inventory_health(product_id)	Returns inventory status for a product: current stock, average daily sales, estimated days to stockout, and a status flag — Critical (<7 days), Low (7–14 days), or Healthy (>14 days).	Compute days_to_stockout = stock_quantity / avg_daily_sales. Flag accordingly. Handle div-by-zero for zero sales.
get_pricing_analysis(product_id)	Returns pricing intelligence: gross margin % (using cost and price columns), price positioning (Premium / Mid-Range / Budget based on category averages), and a flag if margin is below 20%.	Compute gross_margin = (price - cost) / price * 100. Calculate category average price from CSV for positioning.
get_review_insights(product_id)	Uses an LLM to summarise customer reviews for a given product from the reviews CSV. Returns: average rating, total reviews, a 2-sentence sentiment summary, and top 2 recurring themes (positive and negative).	Filter reviews CSV by product_id. Concatenate review texts and pass to LLM for summarisation. Cache results if possible.
get_category_performance(category)	Returns aggregated category-level metrics: total SKUs, average rating, average margin %, total stock units, number of low/critical stock items, and the top 3 revenue-generating products (price × avg_daily_sales).	Aggregate from CSV by category. Return as a structured dict with all metrics clearly labelled.
generate_restock_alert(threshold_days=7)	Scans all products and returns a list of products at risk of stockout within the specified number of days, sorted by urgency (fewest days remaining first). Also includes estimated revenue at risk.	Compute days_to_stockout for all products. Filter by threshold. Sort ascending. Revenue at risk = price × (remaining stock + avg_daily_sales × threshold_days).

Component 2: Router Pattern (Mandatory)

Implement an LLM-powered Router Agent that classifies each incoming user query and dispatches it to the correct downstream handler. Do not use keyword/regex matching — the router must use the LLM for classification.

Route / Intent	Triggered When User Asks...	Handler Behaviour
INVENTORY	About stock levels, stockout risk, restock needs, or how long inventory will last.	Call get_inventory_health() or generate_restock_alert(). Summarise results with urgency labels.

PRICING	About margins, pricing tiers, profitability, cost efficiency, or pricing vs competitors.	Call get_pricing_analysis(). Highlight low-margin items. Suggest pricing action if margin < 20%.
REVIEWS	About customer feedback, ratings, complaints, what customers are saying, sentiment.	Call get_review_insights(). Present sentiment summary and key themes clearly.
CATALOG	About product search, category overviews, top performers, or broad catalog discovery.	Call search_products() or get_category_performance(). Return formatted catalog view.
GENERAL	Greetings, meta questions about the agent, general retail knowledge not in the data.	Respond using LLM knowledge. Use conversation memory if a prior data context exists.

Component 3: Daily Briefing on Startup + Streamlit UI (Mandatory)

When the user launches the app (python run.py), the agent must automatically generate a Daily Briefing before the user types anything. The briefing must include:

- Top 3 most critically low-stock products with days-to-stockout and estimated revenue at risk.
- The worst-rated product in the catalog (lowest average rating) and a one-line summary of why customers are unhappy.
- One pricing flag — the product with the lowest gross margin (if below 25%) and a suggested action.

The Streamlit UI must also include:

- A sidebar with a category filter (All Categories / Tops / Dresses / Bottoms / Outerwear / Accessories) to scope the context.
- A main chat window showing conversation history between the user and the agent.
- An always-visible 'Catalog Summary' panel showing: total SKUs, SKUs with critical stock, average catalog margin %, and average catalog rating.
- A 'Clear Chat' button that resets memory and re-triggers the Daily Briefing.

3. Dataset Description

File 1: retailmind_products.csv (30 rows)		
Column Name	**Data Type**	**Description**
product_id	String	Unique product ID (e.g., SC001)
product_name	String	Full product name (e.g., Floral Summer Dress)

category	String	Tops / Dresses / Bottoms / Outerwear / Accessories
price	Float	Selling price in INR
cost	Float	Cost of goods in INR (for margin calculation)
stock_quantity	Integer	Units currently in inventory
avg_daily_sales	Float	Average units sold per day over the last 30 days
return_rate	Float	Percentage of units returned by customers (0–1 scale)
avg_rating	Float	Average customer rating (1.0–5.0)
review_count	Integer	Total number of reviews on the platform
launch_date	Date	Product launch date (YYYY-MM-DD)
reorder_level	Integer	Stock quantity below which reorder should be triggered

File 2: retailmind_reviews.csv (40 rows)		
Column Name	**Data Type**	**Description**
review_id	Integer	Unique review identifier
product_id	String	Foreign key linking to retailmind_products.csv
reviewer_name	String	Customer's first name
rating	Integer	Star rating given by customer (1–5)
review_title	String	Short title of the review
review_text	String	Full review body text
verified_purchase	Boolean	True if the reviewer actually purchased the item
helpful_votes	Integer	Number of other customers who found this review helpful
review_date	Date	Date review was posted (YYYY-MM-DD)

4. Acceptance Criteria

Acceptance Criterion	Weight	How It Will Be Tested
python run.py launches the Streamlit app without errors. The Daily Briefing appears automatically before any user input.	Pass/Fail	Evaluator runs python run.py cold from repo root.
All 6 tool functions are implemented. The LLM correctly selects the right tool(s) for at least 8 out of 10 test queries.	8/10	Evaluator poses 10 varied queries across all tool types.
Inventory health (days_to_stockout, status flags) is calculated correctly for at least 25 out of 30 products.	25/30	Evaluator manually spot-checks 5 products.
Pricing analysis (gross margin %) is mathematically accurate for all tested products.	Pass/Fail	Evaluator cross-checks 5 products against raw CSV.
Review insights for at least 3 products include a coherent LLM-generated summary and clearly identified positive/negative themes.	3/3	Evaluator queries review insights for 3 products.
Router correctly classifies at least 8 out of 10 intent classification tests.	8/10	Evaluator poses 10 test queries with known correct routes.

5. Submission Guidelines-

Item	Requirement
GitHub Repository	One public/private GitHub repo per student. Name it: creditsage-agent-[your_roll_number]
Entry Point	Evaluator must be able to run: python run.py or python start.py to launch the Streamlit app. No other command should be needed.
<u>README.md</u>	Must include: Project description, pip install -r requirements.txt instructions, .env.example file showing required environment variables, and a brief explanation of your agentic architecture.
Dataset File	The CSV file provided to you at exam start must be placed in the project root and referenced by relative path. Do not hardcode absolute paths.
Commit Window	Your LAST commit must fall within the 2-hour exam window (± 10 minutes). Commits outside this range will not be considered. Time is taken from GitHub's commit timestamp.
API Keys	Never commit API keys. Use .env files and python-dotenv. Include .env.example with variable names but empty values.
Prohibited	No cloud deployment. No external databases. No paid APIs other than OpenAI/Anthropic/Groq. The entire solution must work offline except for LLM API calls.